

1. Relational Model (4 Marks)

The **Relational Model** is a method of organizing data into **tables (relations)** made up of **rows and columns**. Each row is called a **tuple**, and each column is an **attribute**.

It is based on **mathematical set theory** and was proposed by **E.F. Codd**.

Relational databases are easy to use, flexible, and widely supported by database systems like **MySQL, Oracle, and PostgreSQL**.

✅ Example: A table Student (Roll_No, Name, Age) represents a relation with student data.

2. Tuple (4 Marks)

A **tuple** is a **single row** in a relational table. It contains data for **one record** of an entity.

Each tuple includes values for all attributes in the table.

Tuples are **unordered** and each one should be **unique** in a relation unless duplicates are allowed.

✅ Example: In the Student table, this row: (101, Aarti, 20) is a tuple representing one student's data.

3. Attribute (4 Marks)

An **attribute** is a **column** in a table, and it represents a **property** or **characteristic** of an entity.

Every attribute has a **name** and a **data type** (e.g., integer, string, date).

Attributes are used to define the **structure of a table** and are part of the table's schema.

✅ Example: In Student (Roll_No, Name, Age), the attributes are Roll_No, Name, and Age.

4. Domain (4 Marks)

A **domain** is the **set of possible values** an attribute can take. It defines the **valid input** for a column in the table.

Each attribute must have a domain so that invalid values can be avoided.

This helps in **maintaining data accuracy** and **integrity**.

✓ Example: The domain of the Age attribute can be [16 to 25]. Values outside this range are invalid.

5. Schema (4 Marks)

A **schema** defines the **structure** of a relation. It includes the table name and the **list of attributes**.

It helps the database system know what kind of data is stored and how it is organized.

Schema does not contain actual data — it only describes the format.

✓ Example: Student(Roll_No, Name, Age) is a schema showing a table with 3 attributes.

6. Degree (4 Marks)

The **degree** of a relation is the **total number of attributes (columns)** in the table.

It tells how many properties are described for each tuple.

If a table has more columns, its degree is higher.

✓ Example: If a table has attributes Roll_No, Name, Age, then its degree is **3**.

7. Cardinality (4 Marks)

Cardinality is the **number of tuples (rows)** present in a relation.

It tells how many **records** are stored in the table at a particular time.

As data increases, the cardinality also increases.

✅ Example: If a Student table has 10 rows, the cardinality of the relation is **10**.

Sure! Here's the topic "**Keys in DBMS**" in **definition format** for **3 marks each**, written in **easy words** as per PTU exam style:

1. Key in DBMS – Definition (3 Marks):

A **key** in DBMS is a field or combination of fields used to **uniquely identify each record** (row) in a table.

It is used to maintain **data accuracy, integrity**, and to define **relationships between tables**.

2. Primary Key – Definition (3 Marks):

A **primary key** is a column or group of columns that **uniquely identifies each row** in a table.

It **cannot be NULL or duplicate**.

✅ Example: Roll_No in a student table.

3. Candidate Key – Definition (3 Marks):

A **candidate key** is a set of attributes that can **uniquely identify a row** in a table.

Among all candidate keys, one is chosen as the **primary key**.

4. Foreign Key – Definition (3 Marks):

A **foreign key** is an attribute in one table that refers to the **primary key** of another table.

It is used to **create relationships** between two tables and **maintain referential integrity**.

5. Super Key – Definition (3 Marks):

A **super key** is any combination of attributes that **uniquely identifies a row** in a table.

A table can have **many super keys**, but not all are candidate keys.

6. Composite Key – Definition (3 Marks):

A **composite key** is a key formed by combining **two or more columns** to uniquely identify a record.

It is used when a single column is **not enough** to identify records uniquely.

Here's a **3-mark definition format** answer on **Normalization** in DBMS:

Normalization – Definition (3 Marks):

Normalization is the process of organizing the attributes and tables of a **database** to minimize **redundancy** and **dependency**.

It involves dividing large tables into smaller ones and defining relationships between them to improve **data integrity** and **efficiency**.

1st Normal Form (1NF) – Definition (3 Marks):

A table is in **1st Normal Form (1NF)** if it meets the following conditions:

- **All columns contain atomic (indivisible) values.**
- There are **no repeating groups** or arrays in the table.
- Each row has a unique identifier (primary key)

2nd Normal Form (2NF) – Definition (3 Marks):

A table is in **2nd Normal Form (2NF)** if:

- It is in **1st Normal Form (1NF)**.
- It has **no partial dependency**; all non-key attributes are fully dependent on the **primary key**.

3rd Normal Form (3NF) – Definition (3 Marks):

A table is in **3rd Normal Form (3NF)** if:

- It is in **2nd Normal Form (2NF)**.
- It has **no transitive dependency**; non-key attributes do not depend on other non-key attributes.

Boyce-Codd Normal Form (BCNF) – Definition (3 Marks):

A table is in **Boyce-Codd Normal Form (BCNF)** if:

- It is in **3rd Normal Form (3NF)**.
- Every **determinant** is a **candidate key**. In other words, no non-prime attribute should determine any other non-prime attribute.

Sure! Here are the revised answers without the "Conclusion" section:

7. SQL (Structured Query Language) – Definition (15 Marks):

SQL (Structured Query Language) is a domain-specific language used to manage and manipulate relational databases. It provides a set of commands to perform various database operations like creating, modifying, and querying data. SQL is widely supported by most relational database management systems (RDBMS) such as MySQL, Oracle, SQL Server, and PostgreSQL.

Categories of SQL Commands:

SQL commands are broadly categorized into the following types:

1. **DDL (Data Definition Language):**

- a. **CREATE:** Used to create new database objects like tables, views, or schemas.
- b. **ALTER:** Modifies an existing database object, such as adding or deleting columns in a table.
- c. **DROP:** Deletes an entire database object, like a table or view, from the database.

2. DML (Data Manipulation Language):

- a. **SELECT**: Retrieves data from one or more tables based on a query condition.
- b. **INSERT**: Adds new records (rows) to a table.
- c. **UPDATE**: Modifies existing records in a table.
- d. **DELETE**: Removes records from a table based on a condition.

3. DCL (Data Control Language):

- a. **GRANT**: Assigns privileges to a user or role, allowing access to specific database objects.
- b. **REVOKE**: Removes previously granted privileges from a user or role.

4. TCL (Transaction Control Language):

- a. **COMMIT**: Saves all the changes made during the current transaction.
- b. **ROLLBACK**: Reverts changes made during the current transaction, undoing all modifications since the last commit.

8. Relational Algebra – Definition (15 Marks):

Relational Algebra is a procedural query language used to query relational databases. It consists of a set of operations that operate on **relations (tables)** and produce results as relations. These operations help in manipulating and retrieving data from a relational database.

Core Operations in Relational Algebra:

1. **SELECT (σ):**

- a. This operation filters rows based on a specific condition (predicate).
- b. Syntax: **$\sigma_{\text{condition}}(\text{Relation})$**
- c. Example: To find students with a specific age:

$\sigma_{\text{Age}=20}(\text{Student})$

2. **PROJECT (π):**

- a. This operation selects specific columns from a table, eliminating duplicates.
- b. Syntax: **$\pi_{\text{attribute1}, \text{attribute2}, \dots}(\text{Relation})$**
- c. Example: To select only the names and roll numbers of students:

$\pi_{\text{Name}, \text{Roll_No}}(\text{Student})$

3. **UNION ($\cup$):**

- a. Combines two relations with the same number of attributes into a single result, eliminating duplicates.
- b. Syntax: **Relation1 U Relation2**
- c. Example: Combining students from two different departments:

Student_Department1 U Student_Department2

4. SET DIFFERENCE (-):

- a. Finds records that are in one relation but not in another.
- b. Syntax: **Relation1 - Relation2**
- c. Example: Students in department A but not in department B:

DepartmentA - DepartmentB

5. JOIN (⋈):

- a. Combines rows from two relations based on a related column between them.
- b. Types: **Inner Join, Outer Join, Theta Join**, etc.
- c. Example: Joining two tables based on student IDs:

Student ⋈ Department

6. RENAME (ρ):

- a. Renames a relation or attributes.
- b. Syntax: **ρ_new_name(Old_Relation)**
- c. Example: Renaming a student table to Student_Information:

ρ_Student_Information(Student)

9. Functional Dependencies – Definition (15 Marks):

Functional Dependency is a fundamental concept in relational database design that defines the relationship between two attributes in a table. It occurs when one attribute (or set of attributes) **uniquely determines** another attribute.

Concept of Functional Dependency:

- If we say $A \rightarrow B$, it means that attribute **A** determines attribute **B**. This implies that for each value of **A**, there can only be one corresponding value of **B**.
- Functional dependencies are used to maintain the **data integrity** of a relational database and are the basis for database normalization.

Types of Functional Dependencies:

1. Trivial Functional Dependency:

- a. A functional dependency is trivial if the dependent attribute is part of the determinant.
- b. Example: $A \rightarrow A$ is trivial because **A** always determines itself.

2. Non-Trivial Functional Dependency:

- a. A functional dependency is non-trivial if the dependent attribute is not part of the determinant.
- b. Example: $\text{StudentID} \rightarrow \text{StudentName}$, where StudentID determines StudentName.

3. Transitive Functional Dependency:

- a. A functional dependency where one attribute determines a second, and the second attribute determines a third.
- b. Example: If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ is a transitive dependency.

Usage of Functional Dependencies:

- **Normalization:** Functional dependencies are used to decompose large tables into smaller, more efficient ones.
- **Key Determination:** Helps in identifying candidate keys, primary keys, and foreign keys.

10. Transactions and Concurrency Control – Definition (15 Marks):

A **transaction** is a sequence of operations performed as a single unit, typically on a database. A transaction ensures that either **all operations are completed successfully**, or none are applied, maintaining **data integrity**.

ACID Properties of Transactions:

1. Atomicity:

- a. Ensures that all operations in a transaction are completed successfully. If any operation fails, all changes are rolled back to ensure no partial updates are made.
- b. Example: In a bank transaction, money is either **fully transferred** or not transferred at all.

2. Consistency:

- a. Ensures that a transaction takes the database from one valid state to another, maintaining the integrity constraints.
- b. Example: A database should not contain invalid data, such as negative balances in a bank account.

3. Isolation:

- a. Ensures that the operations of one transaction are not affected by the operations of other concurrent transactions.
- b. Example: If two people are transferring money at the same time, they should not interfere with each other's transactions.

4. Durability:

- a. Ensures that once a transaction is committed, its effects are permanent, even in the event of system failures.
- b. Example: After committing a bank transfer, the change is saved permanently, even if the system crashes.

Concurrency Control:

Concurrency control is a set of techniques used to ensure that database transactions are executed in a way that preserves **data consistency**, even when multiple transactions are being processed simultaneously by multiple users. It deals with problems like **lost updates**, **temporary inconsistencies**, and **uncommitted data**.

- **Locking Mechanism:** Prevents multiple transactions from accessing the same data at the same time.
- **Optimistic Concurrency Control:** Transactions are allowed to execute without locking data, and conflicts are detected later.

Let me know if you need any further clarification!